

Державний вищий навчальний заклад
Ужгородський національний університет
Факультет інформаційних технологій
Кафедра програмного забезпечення систем

ЛАБОРАТОРНА РОБОТА № 4

Тема: Апроксимація функцій методами інтерполяції та МНК

Виконав:
студент II курсу
спеціальності: 121 інженерія
програмного забезпечення
Романюк Артем Романович

Ужгород-2026

Мета: засвоїти підходи до апроксимації даних, зокрема інтерполяційний поліном Лагранжа та метод найменших квадратів, а також набути практичних навичок їх реалізації мовою програмування JavaScript і візуалізації отриманих результатів.

Завдання до роботи:

1. Розробка технічного завдання (ТЗ): Опис функціональних вимог, інтерфейсу та обмежень системи.
2. Вибір методології: Визначити підхід до розробки (наприклад, Agile, Spiral або V-model і т.д.). Важливо: обрана методологія має відрізнятися від тих, що використовувалися у ЛР № 1, 2 та 3.
3. Проектування логіки: Створити графічний алгоритм (блок-схеми) та скласти таблиці ідентифікаторів (змінних, функцій, об'єктів).
4. Програмна реалізація: Реалізувати код відповідно до ТЗ, обраного стеку та методології.
5. Аналітичне тестування: Провести тести та порівняльний аналіз результатів для різної кількості даних.
6. Моделювання взаємодії: Побудувати діаграми взаємодії, що відображає роботу компонентів ПЗ.
7. Документування: Оформити звіт та завантажити код в репозиторій GitHub з коментуванням етапів відповідно до методології розробки ПЗ, алгоритму та пункту 8.
8. Звіт про використання AI (AI Interaction Log):
Визначити рівень залучення штучного інтелекту до розробки та надати докази взаємодії.

Хід роботи

1. Розробка технічного завдання

Технічне завдання

Зміст:

1. Загальна інформація
2. Мета проекту
3. Основні функціональні вимоги

1. Загальна інформація

Назва проекту: Програмний модуль для апроксимації функцій методами інтерполяції та МНК.

Галузь застосування: Навчальний процес дисципліни «Методи програмної інженерії для розв'язування прикладних задач».

Цільова платформа: Веб-середовище (браузер).

Інструментарій розробки:

- Мова програмування: JavaScript.
- Розмітка та стилі: HTML5, CSS3.
- Візуалізація: бібліотека D3.js.

2. Мета проекту

Метою розробки є створення програмного продукту, який дозволяє:

- Реалізувати побудову інтерполяційного полінома Лагранжа за заданим табличним набором точок $\{(x_i, y_i)\}$.
- Виконати апроксимацію даних методом найменших квадратів (МНК) із вибором ступеня полінома $m=2, 3, 4$.
- Візуалізувати результати: відобразити вихідні точки, криву інтерполяції та криву МНК, а також діаграму залишків r_i .
- Забезпечити анімацію покрокового додавання вузлів інтерполяції та «виростання» кривої МНК.

3. Основні функціональні вимоги

Організація інтерфейсу: адаптивне полотно D3.js, перемикач «Інтерполяція / МНК / Усі», осі з підписами й сіткою, легенда, підказки при наведенні (координати, значення похибки).

Введення даних: можливість введення набору точок $\{(x_i, y_i)\}$ через форму або вибір варіанта зі списку.

Алгоритмічне забезпечення:

- Інтерполяція Лагранжа - точне відтворення значень функції в усіх заданих вузлах.
- МНК-апроксимація - побудова дизайн-матриці, обчислення коефіцієнтів через QR-розклад, обчислення залишків r_i , RMSE, σ^2 .

Обробка та вивід результатів:

- Відображення графіка функції з точками, кривою інтерполяції, кривою МНК.
- Побудова діаграми залишків (вертикальні відрізки від точки до кривої МНК).
- Обчислення та відображення метрик якості: RMSE, R^2 .

Анімація:

- Інтерполяція: покрокове додавання вузлів із плавним оновленням кривої.
- МНК: плавна поява полінома.

Вимоги до коду: Git-коміт для кожного логічного етапу з описовим повідомленням.

2. Вибір та опис методології розробки ПЗ. Розробка методики вирішення задачі

Для реалізації проекту обрано каскадну методологію Waterfall. На відміну від ітеративних підходів (Scrum, Kanban), Waterfall передбачає чітко визначені послідовні фази, кожна з яких повністю завершується перед переходом до наступної. Для даного проекту це є доцільним, оскільки вимоги до алгоритмів інтерполяції та МНК є чітко сформульованими у технічному завданні та не змінюватимуться в процесі розробки.

Фази Waterfall для даного проекту:

Фаза	Зміст робіт	Результат
1. Аналіз вимог	Опис вхідних даних $\{(x_i, y_i)\}$, визначення методів (Лагранж, МНК), формулювання	Затверджене ТЗ

	функціональних і нефункціональних вимог	
2. Проектування	Проектування архітектури модулів: введення даних, обчислення, візуалізація. Розробка діаграми взаємодії компонентів	Діаграма компонентів, структура файлів
3. Реалізація	Розробка модулів у послідовності: введення даних → інтерполяція Лагранжа → МНК → візуалізація D3.js → анімація	Програмний код JS, HTML, CSS
4. Тестування	Перевірка коректності алгоритмів на тестових наборах, порівняння результатів із аналітичними значеннями, перевірка метрик RMSE/R ²	Протокол тестування
5. Розгортання	Завантаження на GitHub, оформлення звіту, фінальний коміт	Репозиторій, звіт

3. Проектування логіки

Інтерполяція Лагранжа

За $n+1$ вузловими точками (x_i, y_i) будується поліном ступеня n , що точно проходить через всі задані точки:

$$P(x) = \sum y_k \cdot L_k(x), \text{ де } L_k(x) = \prod (x - x_j) / (x_k - x_j), \quad j \neq k$$

Перевага: проста реалізація.

Недолік: при великій кількості вузлів виникає ефект Рунге (осциляція на краях).

Апроксимація МНК

На відміну від інтерполяції, МНК не вимагає точного проходження через вузли, а мінімізує суму квадратів відхилень.

Алгоритм:

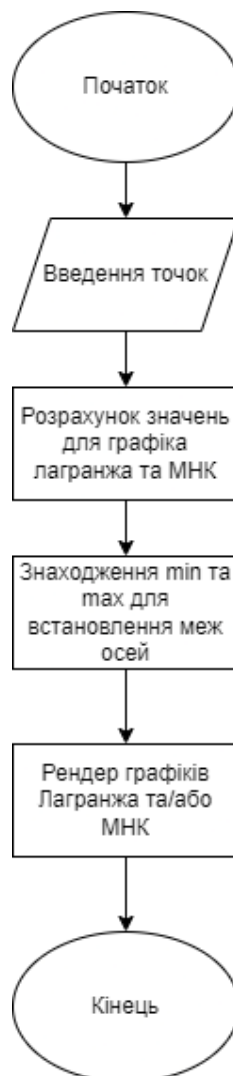
Побудувати дизайн-матрицю Φ розміру $(n \times (m+1))$, де m — обраний ступінь полінома.

Знайти вектор коефіцієнтів $\theta = (\Phi^T \Phi)^{-1} \Phi^T y$ (через QR-розклад для числової стійкості).

Обчислити залишки $r_i = y_i - \hat{y}_i$ та метрики $RMSE = \sqrt{(\sum r_i^2 / n)}$, $R^2 = 1 - \sum r_i^2 / \sum (y_i - \bar{y})^2$.

Вибір ступеня m : при малому m - недоапроксимація, при $m \geq n-1$ — вироджується в інтерполяцію та можливе перенавчання.

Блок-схема:



Таблиця ідентифікаторів:

Ідентифікатор	Тип	Призначення
---------------	-----	-------------

xs, ys	number[]	Масиви координат вузлових точок
x	number	Точка в якій рахується $P(x)$
n	number	Кількість вузлів
coeffs	number[]	Коефіцієнти $a_0, a_1, \dots$ полінома
Phi	number[][]	Дизайн-матриця МНК
residuals	number[]	Залишки $r_i = y_i - \hat{y}_i$
rmse, r2	number	Метрики якості МНК
lagrange()	(number[], number[]) $\rightarrow f_n$	Поліном Лагранжа
leastSquares()	(number[], number[], number) $\rightarrow \{f_n, residuals, rmse, r2\}$	МНК-апроксимація через QR
App	React FC	Кореневий компонент, зберігає стан
Chart	React FC	D3.js візуалізація кривих
InputPanel	React FC	Введення вузлових точок
MetricsPanel	React FC	Відображення RMSE, R^2 , залишків

4. Програмна реалізація

5. Аналітичне тестування

6. Моделювання взаємодії

Діаграма відображає послідовність взаємодії між компонентами при виконанні інтерполяції та МНК:

Користувач	UI (HTML/CSS)	Controller (JS)	D3.js Renderer
------------	---------------	-----------------	----------------

Вводить точки $\{(x_i, y_i)\}$	→	Форма / Select варіанта				
		Валідація вхідних даних	→	lagrange() / ls()		
				Обчислення коефіцієнтів, залишків, RMSE, R^2	→	drawChart(points, curve, residuals)
						Анімація кривої + залишки на SVG
Бачить графік, метрики, анімацію	←	Оновлення UI (перемикач, легенда, tooltip)	←	Повертає результати	←	

7. Документування

8. Звіт про використання AI (AI Interaction Log)

Висновки